

MemGrid

Distributed Shared Memory Management System

SUBMITTED BY

Jui Sudhir Tawde

Roll Number: 150096725149

Class / Division: Sam Altman

Course: Data Structures and Algorithms using C++

Batch: BTECH-CSE

Academic Year: 2025–2026

INSTITUTION

ITM Skills University, Kharghar

3.3 Project Title

MemGrid – Distributed Shared Memory Management System

3.4 Problem Statement

MemGrid is a Distributed Shared Memory (DSM) management system that allows multiple computers in a supercomputer cluster to share a unified memory space. Although memory blocks are physically distributed across different machines, the system provides a logical view of shared memory.

The project manages computers, memory blocks, network connections, memory allocation, shortest path routing, rollback operations, and memory access tracking using efficient Data Structures and Algorithms.

The system demonstrates how distributed systems can organize and access memory efficiently while maintaining fast search, allocation, and communication between nodes.

3.5 Objectives

1. To simulate a Distributed Shared Memory environment.
2. To manage memory blocks across multiple computers.
3. To maintain a balanced memory index using an AVL Tree.
4. To verify and locate computers using Hash Tables.
5. To process memory requests using Queue data structure.
6. To support rollback operations using Stack.
7. To rank frequently accessed memory blocks using Heap.
8. To model network topology using Graphs.
9. To determine shortest communication paths using Dijkstra's Algorithm.
10. To provide efficient memory allocation and management.

3.6 Design / Architecture

System Components

- Computer Management Module – Stores cluster computers, maintains memory capacities, uses Hash Table for fast lookup.
- Memory Block Management Module – Stores memory blocks, tracks location and size, uses AVL Tree as balanced index.

- Request Processing Module – Handles incoming memory requests; implemented using Queue.
- Rollback Module – Stores previous operations and allows undo functionality; implemented using Stack.
- Access Tracking Module – Tracks block access frequencies using Max Heap for ranking.
- Network Management Module – Represents cluster topology; implemented using Graphs.
- Routing Module – Computes shortest communication paths; uses Dijkstra's Algorithm.

Architecture Flow

User Request → Memory Request Queue → AVL Memory Index Search → Computer Allocation → Graph Network Routing → Result Returned

3.7 Algorithm Description

1. AVL Tree

Maintains a balanced index of all memory blocks, guaranteeing $O(\log n)$ performance for insert, search, and delete through automatic rotations.

- Insert Memory Block
- Search Memory Block
- Delete Memory Block

2. Hash Table

Stores and verifies computers with average $O(1)$ lookup time for instant computer identification.

- Insert Computer
- Search Computer

3. Queue

Processes memory requests in strict FIFO order, ensuring every request is handled fairly.

- Enqueue Request
- Dequeue Request

4. Stack

Stores operation history enabling rollback / undo of the most recent memory move.

- Push Operation
- Pop Operation

5. Max Heap

Identifies and ranks the most frequently accessed memory blocks with efficient $O(\log n)$ priority handling.

- Insert Frequency Data
- Retrieve Highest Frequency Block

6. Graph

Represents the full computer network topology and models all communication links between cluster nodes.

- Add Connections
- Display Network

7. Dijkstra's Algorithm

Finds the shortest communication path between any two computers in the cluster network.

1. Initialize source distance as 0; mark all other nodes as infinity.
2. Select the nearest unvisited node.
3. Update neighboring distances if a shorter path is found.
4. Repeat until destination is reached.

3.8 Complexity Analysis

Data Structure / Algorithm	Operation	Time Complexity
Hash Table	Search / Insert	$O(1)$ Average
AVL Tree	Search / Insert / Delete	$O(\log n)$
Queue	Enqueue / Dequeue	$O(1)$
Stack	Push / Pop	$O(1)$
Heap	Insert / Extract Max	$O(\log n)$
Graph	Add Edge	$O(1)$
Dijkstra Algorithm	Shortest Path	$O((V + E) \log V)$

Where V = Number of Computers and E = Number of Network Connections between them.

3.9 Screenshots of Execution

```
juitawde@Juis-MacBook-Air dsa_final % g++ system.cpp -o system
juitawde@Juis-MacBook-Air dsa_final % ./system

MemGrid

=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : █
```

Fig 1: MemGrid startup – ASCII art banner and main menu

```
juitawde@Juis-MacBook-Air dsa_final % ./system
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 1
Computer ID : 101
Total Memory : 1000
Computer Added Successfully
```

Fig 2: Add Computer – Computer 101 added with 1000 MB memory

```
juitawde@Juis-MacBook-Air dsa_final % ./system
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 2
Enter Computer ID : 101

Computer Found
ID : 101
Free Memory : 1000

=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : █
```

Fig 3: Verify Computer – Computer 101 found with 1000 MB free

```
juitawde@Juis-MacBook-Air dsa_final % ./system
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 3
Block ID : 10
Block Size : 200

Allocated To Computer 101
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : █
```

Fig 4: Allocate Memory Block – Block 10 (200 MB) allocated to Computer 101

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juitawde@Juis-MacBook-Air dsa_final % ./system
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 15

Cluster Computers
ID : 104
Memory : 300
Free : 300
ID : 103
Memory : 4000
Free : 4000
ID : 102
Memory : 3000
Free : 3000
ID : 101
Memory : 1000
Free : 800
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
```

Fig 5: Display Memory Blocks – AVL Tree in-order listing of all blocks

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juitawde@Juis-MacBook-Air dsa_final % ./system
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 5

Memory Index
Block ID : 10 | Size : 200 | Stored At : 101
Block ID : 11 | Size : 10 | Stored At : 104
Block ID : 13 | Size : 800 | Stored At : 101
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
```

Fig 6: Display Computers – All cluster computers with memory details

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juitawde@Juis-MacBook-Air dsa_final % ./system
15. Display Computers
16. Exit
Choice : 6
Block ID : 11
New Computer ID : 103
Block Moved Successfully
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 7

Rollback Successful
Block 11 restored to Computer 101
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
```

Fig 7: Move Memory Block and Rollback – Block moved to 103, restored to 101 via Stack

```

juitawde@Juis-MacBook-Air dsa_final % ./system
Choice : 8
Request ID : 1
Computer ID : 102
Block ID : 11
Request Added
=====
MemGrid Menu
=====
1. Add Computer
2. Verify Computer
3. Allocate Memory Block
4. Search Memory Block
5. Display Memory Blocks
6. Move Memory Block
7. Undo Last Move
8. Add Request
9. Process Request
10. Access Memory Block
11. Show Most Used Blocks
12. Add Network Connection
13. Display Network
14. Find Shortest Path
15. Display Computers
16. Exit
Choice : 9

Processing Request
Request ID : 1
Computer ID : 102
Block ID : 11

Access Granted
Computer 102 accessed Block 11
Block Stored At Computer 101
Access Frequency : 1
=====
MemGrid Menu
=====

```

Fig 8: Add Request and Process Request – Queue-based memory access with Access Granted

3.10 Results and Findings

- The system successfully simulated a Distributed Shared Memory environment.
- Memory blocks were efficiently indexed using AVL Trees.
- Computer verification was performed quickly using Hash Tables.
- Memory requests were processed correctly using Queue (FIFO order).
- Rollback functionality worked efficiently using Stack.
- Frequently accessed blocks were tracked and ranked using Max Heap.
- Network topology was represented and traversed using Graphs.
- Dijkstra's Algorithm successfully determined shortest communication paths.
- The project demonstrated practical applications of multiple data structures working together in a single unified system.

3.11 Conclusion

MemGrid successfully demonstrates the implementation of a Distributed Shared Memory Management System using various Data Structures and Algorithms. The project integrates AVL Trees, Hash Tables, Queues, Stacks, Heaps, Graphs, and Dijkstra's Algorithm to efficiently manage memory allocation, request processing, routing, and rollback operations.

The system provides a realistic simulation of how distributed computing environments organize and access shared memory while maintaining efficiency and scalability. The project also

highlights the importance of selecting appropriate data structures for specific tasks in large-scale computing systems.

3.12 GitHub Repository

GitHub: <https://github.com/juitawde/MemGrid.git>